

A 1.2.4 Truth Tables

	AB			
	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	0	0	0	1
10	0	1	1	1

A 1.2.4 Construct and analyse truth tables

A truth table lists every possible combination of inputs to a logic system and records the output for each one. Because it is exhaustive, it settles questions that intuition cannot: two circuits that look completely different are equivalent exactly when their truth tables match, row for row. This page shows how to build a truth table from a circuit and from a worded problem, how a table and a Boolean expression describe the same function in both directions, and how to derive a table from a logic diagram in order to simplify it. Karnaugh maps and the laws of Boolean algebra are the two routes to that simplification, and both are examinable.

Last Updated:: 12 June 2026

Key Questions

- How many rows does a truth table need for n inputs, and how do you lay them out without missing any?
- Given a logic circuit, how do you work out its truth table?
- How do you read a Boolean expression off a truth table, and how do you go back the other way?
- How do you build a Karnaugh map for two or three variables, and what are the rules for grouping?
- When is an expression fully simplified?

Scope Note

This syllabus limits logic diagrams and Karnaugh maps to a maximum of three variables. Every circuit, truth table and map on this page therefore uses at most three inputs. You will not be asked to draw a four-variable map or read a logic diagram with more than three inputs, so the techniques below are shown in full at the scale you will actually meet.

Throughout the page the notation is standard: $A \cdot B$ means A AND B , $A + B$ means A OR B , and $\bar{A}$ (a bar over the letter) means NOT A , the complement of A .

IB COMMAND TERM NOTE

This learning statement uses two command terms.

Construct means produce the table in full: every input combination present, in a consistent order, with a clear output column. A table missing rows, or with inputs in a haphazard order, will not score even if the rows shown are correct.

Analyse means break the table down to bring out its structure: derive the Boolean expression it represents, or use it to find a simpler equivalent. For "analyse" questions the examiner expects working, not just a final expression. Show the product terms you read off, or the Karnaugh map groups you formed.

Building a truth table

A **truth table** has one column per input, optional columns for intermediate signals, and one column for the output. The number of rows is fixed by the number of inputs: each input can be 0 or 1, so n inputs give 2^n distinct combinations. One input gives 2 rows, two inputs give 4, three give 8.

The reliable way to fill the input columns is by following the method below, not by guessing combinations as they occur to you. For three inputs A, B, C, work column by column:

- The right-hand input C alternates every row: 0, 1, 0, 1, 0, 1, 0, 1.
- The middle input B alternates in pairs: 0, 0, 1, 1, 0, 0, 1, 1.
- The left-hand input A alternates in fours: 0, 0, 0, 0, 1, 1, 1, 1.

Read down the three columns and you have counted from 000 to 111 in binary, with all eight rows present and none repeated:

A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

A	B	C
1	1	1

The same method works for two inputs: drop column A and you are left with the four rows 00, 01, 10, 11. Set this skeleton down first, then work out the output for each row.

The operators you tabulate are the building blocks. You need their behaviour to read any circuit:

Operator	Notation	Output is 1 when
AND	$A \cdot B$	both inputs are 1
OR	$A + B$	at least one input is 1
NOT	$\bar{A}$	the single input is 0
XOR	$A \oplus B$	the two inputs differ

COMMON EXAM MISTAKE

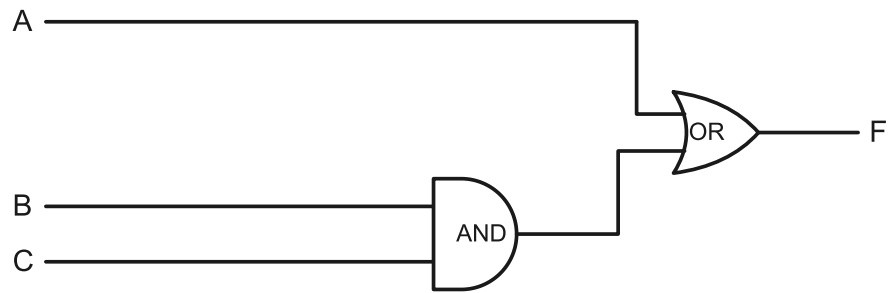
Writing $2n$ rows instead of 2^n , or listing the input combinations in an inconsistent order and accidentally repeating or omitting one. For three inputs the table has eight rows, not six. Lay down the binary-count skeleton above and the rows take care of themselves.

Predicting the output of a logic circuit

To find a circuit's truth table, trace each signal from the inputs to the output, one gate at a time.

Add a column for every gate output, fill those columns first, then combine them for the final output.

The circuit below computes $A + (B \cdot C)$: inputs B and C feed an AND gate, and its output is combined with A in an OR gate.



A two-gate circuit computing $F = A + (B \cdot C)$. Alt text: Logic circuit with inputs A, B and C. B and C enter an AND gate; the AND output and A enter an OR gate, producing output F.

Work the intermediate column $B \cdot C$ first, then OR it with A.

WORKED EXAMPLE

Truth table for $F = A + (B \cdot C)$:

A	B	C	$B \cdot C$	F
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Notice the pattern in F. Whenever A is 1, F is 1, whatever B and C do.

F is also 1 in the single row where A is 0 but B and C are both 1.

Hold that observation; it is exactly what the simplification later recovers.

Building a truth table from a problem description

Often you are not given a circuit but a description in words. The task is to name the inputs and output, translate the conditions into operators, then build the table. Consider a greenhouse fan that should switch on when the temperature is high *and* either the humidity is high *or* the manual override is set.

Let T, H and M be the three conditions (1 means the condition holds) and let F be the fan. "High temperature AND (high humidity OR override)" maps directly to $F = T \cdot (H + M)$.

T	H	M	$H + M$	F
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

The fan stays off whenever the temperature is low, because the AND requires T to be 1. That matches the description, which is the check you should always make: read the table back as a sentence and confirm it says what the problem said.

COMMON EXAM MISTAKE

Reading "either X or Y" as exclusive when the problem means inclusive. In everyday speech "or" sometimes means one but not both; in a problem like the one above it means one *or the other or both*, which is OR, not XOR. Decide which the wording requires before you build the table, and state your assumption if the wording is genuinely ambiguous.

Truth tables and Boolean expressions

A truth table and a Boolean expression are two descriptions of the same function, and you can move between them in either direction.

To go from a table to an expression, look at each row where the output is 1. Write a product (AND) of all the inputs, using the variable itself where it is 1 and its complement where it is 0. Join those products with OR. The result is a sum-of-products expression, and it reproduces the table exactly by construction.

To go the other way, from an expression to a table, take each row of inputs in turn, substitute the 0s and 1s into the expression, and evaluate it to fill the output column. The two processes are inverses of each other.

Take the table from the circuit above. The output is 1 in five rows, so the sum-of-products expression has five product terms:

A	B	C	F	Product term
0	1	1	1	$\bar{A} \cdot B \cdot C$
1	0	0	1	$A \cdot \bar{B} \cdot \bar{C}$
1	0	1	1	$A \cdot \bar{B} \cdot C$
1	1	0	1	$A \cdot B \cdot \bar{C}$
1	1	1	1	$A \cdot B \cdot C$

So $F = \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot \bar{C} + A \cdot \bar{B} \cdot C + A \cdot B \cdot \bar{C} + A \cdot B \cdot C$.

From a logic diagram to a simpler circuit

The expression above is correct but long. Five product terms means a circuit of several AND gates feeding a wide OR gate, when the original used two gates. This is the standard workflow when you are handed a logic diagram and asked to simplify it: derive the truth table from the diagram, read the sum-of-products expression off the table, then reduce that expression. The table is the bridge between a circuit you can see and an expression you can manipulate, and a simpler expression helps to build a simpler circuit.

THINK FIRST

Look at $F = \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot \bar{C} + A \cdot \bar{B} \cdot C + A \cdot B \cdot \bar{C} + A \cdot B \cdot C$ before reading on. Four of the five terms begin with

A. What happens if you factor A out of those four, and what does the bracket that is left behind reduce to?

Algebraic simplification

Boolean algebra has a small set of laws that let you rewrite an expression into a shorter equivalent. The ones used most in simplification are:

- **Complement:** $X + \bar{X} = 1$ and $X \cdot \bar{X} = 0$
- **Identity:** $X \cdot 1 = X$ and $X + 0 = X$
- **Distribution:** $X \cdot (Y + Z) = X \cdot Y + X \cdot Z$
- **Absorption:** $X + X \cdot Y = X$
- **De Morgan:** the complement of a product is the sum of the complements, $\text{NOT}(X \cdot Y) = \bar{X} + \bar{Y}$, and the complement of a sum is the product of the complements, $\text{NOT}(X + Y) = \bar{X} \cdot \bar{Y}$

WORKED EXAMPLE

Simplify $F = \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot \bar{C} + A \cdot \bar{B} \cdot C + A \cdot B \cdot \bar{C} + A \cdot B \cdot C$.

Factor A from the last four terms:

$F = \bar{A} \cdot B \cdot C + A \cdot (\bar{B} \cdot \bar{C} + \bar{B} \cdot C + B \cdot \bar{C} + B \cdot C)$ [1] group the A terms

Simplify inside the bracket using the complement law:

$$\bar{B} \cdot \bar{C} + \bar{B} \cdot C = \bar{B} \cdot (\bar{C} + C) = \bar{B} \cdot 1 = \bar{B}$$

$$B \cdot \bar{C} + B \cdot C = B \cdot (\bar{C} + C) = B \cdot 1 = B$$

so the bracket is $\bar{B} + B = 1$ [1] bracket reduces to 1

Substitute back, using $X \cdot 1 = X$:

$$F = \bar{A} \cdot B \cdot C + A \cdot 1 = \bar{A} \cdot B \cdot C + A$$
 [1] identity law

Apply the rule $A + \bar{A} \cdot X = A + X$ (here X is $B \cdot C$):

$$F = A + B \cdot C$$
 [1] final simplified expression

The five-term expression and the two-term expression produce identical truth tables. They are the same function written two ways, and $F = A + B \cdot C$ is exactly the two-gate circuit shown at the top of the page.

Karnaugh maps

Algebra always works, but it relies on spotting the right law at the right moment, and a mistake is easy to miss. A **Karnaugh map** reaches the same answer by eye. It is the same **truth table** redrawn as a grid, arranged so that any two cells next to each other differ in exactly one variable. That arrangement is the whole point: when adjacent 1s are grouped, the variable that changes within the group cancels, leaving a shorter term.

Gray code ordering

The single-variable-change property only holds if the rows and columns are labelled in the right order. The labels run 00, 01, 11, 10, **not** 00, 01, 10, 11.

This is Gray code: each label differs from its neighbour in one bit only.

Moving from column 01 to column 11 changes one bit; moving from 11 to 10 changes one bit; and because the map wraps, moving from 10 back round to 00 also changes one bit. Label the axes wrongly and the adjacencies break, so the groups stop meaning anything.

Two-variable map

With two inputs there are four cells, one variable on each axis. Take $F = A \cdot \bar{B} + A \cdot B$:

		A	
		0	1
B	0	0	1
	1	0	1

$A \cdot \bar{B} + A \cdot B$

The two 1s sit above each other in the $A = 1$ column.

		A	
		0	1
B	0	0	1
	1	0	1

$A \cdot \bar{B} + A \cdot B = A$

Across that group B changes from 0 to 1 while A stays at 1, so B drops out and the group reads simply A. Therefore $F = A$.

A group of two has removed one variable.

Three input Karnaugh Maps

For a Karnaugh map with three inputs, one row or column represents two of them.

Consider a map for three inputs A, B, C.

The map for this expression is shown to the right. In this map:

- There are four rows to represent the possible values of the inputs AB
- There are two columns to represent the possible values of the input C

		C	
		0	1
AB	00		
	01		
	11		
	10		

Flexible Input Arrangement:

- The number of rows and columns in a Karnaugh map is not fixed.
- Inputs can be split across rows and columns in various ways. (BC, A etc.)

Sequential Input Value Changes:

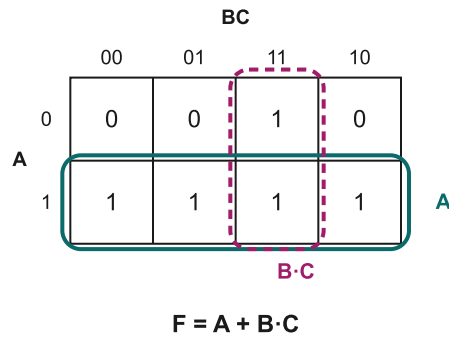
- Only one input value should change between adjacent cells within a row or column.
- For example, if one cell has input values "01," the next cell could be "11" but not "10."

Non-Standard Row Order:

- The arrangement of rows in a Karnaugh map differs from a traditional truth table.
- This specific order is essential for grouping adjacent cells with similar output values.

Three-variable map example

With three inputs, this time lets put one variable on the rows (A) and the remaining two on the columns (BC) in Gray-code order. The eight cells hold the eight outputs. Returning to the function from the circuit, whose 1-rows we listed above:



Grouping the ones for F. The group of four (whole $A = 1$ row) gives A; the group of two ($BC = 11$ column) gives $B \cdot C$, so $F = A + B \cdot C$.

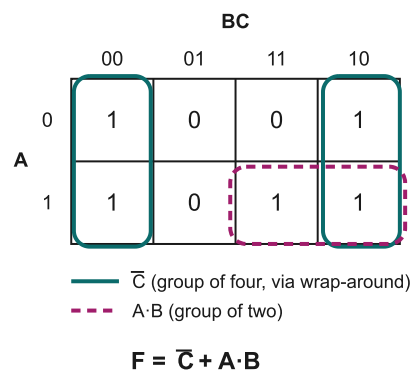
Read the map in groups. The bottom row is all 1s and shares only $A = 1$, so that group of four is simply A.

The two 1s in the $BC = 11$ column share $B = 1$ and $C = 1$ across both values of A, so that group of two is $B \cdot C$.

Combining the groups gives $F = A + B \cdot C$, the same answer the algebra produced and the same circuit shown at the top of the page.

A three-variable map that uses every trick

The first map needed only a plain row and a plain column. A three-variable map can also call for grouping across the edges and for groups that overlap, and you reach all of those without ever leaving three variables. Take $F = \bar{C} + A \cdot B$, mapped from its truth table:



A three-variable map using wrap-around and overlap. The outer columns 00 and 10 share $C = 0$, so together they form one group of four giving $\bar{C}$; the two ones in the bottom row of columns 11 and 10 give $A \cdot B$. So $F = \bar{C} + A \cdot B$.

Three things are worth drawing out from this map:

- **A group of four.** The outer two columns, 00 and 10, both have $C = 0$. The four 1s they contain form a single group of four, and across it both A and B change while only C stays fixed, so the group reads $\bar{C}$. A group of four has removed two variables.
- **Wrap-around.** Columns 00 and 10 are not side by side on the page, but in Gray code they differ in one bit, so the map treats the left and right edges as joined. The group is legitimate even though it looks split.
- **Overlap.** The bottom-right cell belongs both to the $\bar{C}$ group and to the $A \cdot B$ group. A 1 may be used in more than one group, and using it twice here is what lets both groups grow to their full size.

Grouping rules

The rules are fixed, and the same set applies to two- and three-variable maps:

- Group only 1s, never 0s.
- Every group's size is a power of two: 1, 2, 4 or 8. In a three-variable map a group of 8 covers every cell, which means the output is always 1.
- Groups are rectangular in the wrapped sense; an L-shape or a group of three is never allowed.
- Make each group as large as the rules allow. A group of 2 drops one variable, a group of 4 drops two.
- Groups may overlap. A single 1 may belong to more than one group.
- Groups may wrap around the left and right edges (and, in a map drawn with two rows, the top and bottom).
- Cover every 1 at least once, using as few groups as possible.
- Read each group as the product of the variables that stay constant across it; ignore the variables that change.

EXAM TIP

Always take the largest legal group, and remember that groups can wrap around the edges and can overlap. If your simplified expression still has more terms than the map suggests, you have probably split a group that could have been merged, or missed a wrap-around across the outer columns.

COMMON EXAM MISTAKE

Forming a group of three cells, or stopping once every 1 is covered without checking whether the groups could be larger. Group sizes must be powers of two, and groups smaller than necessary leave the expression only partly simplified.

Summary reference

Idea	What to do
Number of rows	n inputs give 2^n rows; fill input columns by the binary-count rhythm (rightmost alternates every row, next in pairs, next in fours)
Predict a circuit's output	Add a column per gate, fill the intermediate signals, then the output
From a problem description	Name inputs and output, translate the wording into operators, build the table, read it back
Table to expression	One product term per 1-row; variable if 1, complement if 0; OR the terms together (sum of products)
Expression to table	Substitute each input combination into the expression and evaluate
Algebraic simplification	Apply complement, identity, distribution, absorption and De Morgan's laws
Karnaugh map	Label axes in Gray code (00, 01, 11, 10); group adjacent 1s in powers of two, as large as possible; overlaps and wrap-around allowed; each group drops the variables that change within it
Which method	A map is quickest for the two or three variables this syllabus uses; algebra always works and is the fallback when a map is awkward
When is it done?	Every 1 covered by a maximal group; no group can be enlarged or merged

TRY IT ON PAPER

The site quiz below covers recognition and recall. Attempt each before reading the model answer.

1. Construct the complete truth table for $F = (A + B) \cdot \bar{C}$. **[2 marks]**

Model answer:

A	B	C	$A + B$	$\bar{C}$	F
0	0	0	0	1	0
0	0	1	0	0	0

A	B	C	A + B	$\bar{C}$	F
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	0	0
1	1	0	1	1	1
1	1	1	1	0	0

All eight input combinations present and correctly ordered [1]. $\bar{C}$ inverted and combined correctly, so F is 1 only in the three rows where C = 0 and at least one of A, B is 1 [1].

2. The function $F = A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B$ is read off a truth table. Simplify it as far as possible, showing your working, and state the result. **[4 marks]**

Model answer:

$A \cdot B + A \cdot \bar{B} = A \cdot (B + \bar{B}) = A \cdot 1 = A$ [1] for factoring A, [1] for applying complement then identity.

This leaves $F = A + \bar{A} \cdot B$. Applying $A + \bar{A} \cdot X = A + X$ gives $F = A + B$ [1] for the absorption step.

Final answer: $F = A + B$ [1].

3. Complete the definition (cloze). **[no marks; self-check]**

A truth table for n inputs has _____ rows. To read a sum-of-products expression from it, take the _____ of the inputs for every row whose output is _____, using the variable when it is 1 and its _____ when it is 0, then take the _____ of those terms. On a Karnaugh map the axes are labelled in _____ code so that adjacent cells differ in one variable.

Model answer: 2^n ; product (AND); 1; complement (NOT); sum (OR); Gray.

A 1.2.4 Quiz

1

A logic system has 3 inputs. How many rows does its complete truth table need?

A. ☐ 3

B. ☐ 6

C. ☒ 8

D. ☐ 9

Each input doubles the number of combinations, so the count is $2^3 = 8$. The wrong answer 6 comes from calculating 2×3 instead of 2 to the power 3.

2

For inputs A and B, an output is 1 only when both inputs are 1. Which operator is this?

A. ☐ $A + B$

B. ☒ $A \cdot B$

C. ☐ $\bar{A}$

D. ☐ $A \oplus B$

"Both inputs 1" is the definition of AND. OR ($A + B$) would also be 1 when just one input is 1, and XOR is 1 only when the inputs differ, so both are 0 when $A = B = 1$.

3

In a truth table, the row $A = 1, B = 0, C = 1$ has output 1. Which product term represents this row in a sum-of-products expression?

A. ☐ $A \cdot B \cdot C$

B. ☒ $A \cdot \bar{B} \cdot C$

C. ☐ $\bar{A} \cdot B \cdot \bar{C}$

D. ☐ $A + \bar{B} + C$

Each variable appears as itself when 1 and as its complement when 0, so $A = 1$ gives A, $B = 0$ gives $\bar{B}$, $C = 1$ gives C: $A \cdot \bar{B} \cdot C$. Option A forgets to complement B; option D uses OR, which is

the product-of-sums form, not sum of products.

4

A truth table has output 1 in exactly three rows. Before any simplification, how many product terms does its sum-of-products expression contain?

A. ☐ 1

B. ☐ 2

C. ☒ 3

D. ☐ 8

The sum-of-products expression has one product term for each row whose output is 1, so three 1-rows give three terms.

5

On a Karnaugh map, which ordering is correct for the two-variable axis?

A. ☐ 00, 01, 10, 11

B. ☒ 00, 01, 11, 10

C. ☐ 0, 1, 2, 3

D. ☐ 00, 10, 01, 11

The axis uses Gray code, in which each label differs from its neighbour in exactly one bit. That is what makes physically adjacent cells differ in a single variable, which is the property grouping relies on.

6

Two adjacent cells of a Karnaugh map both hold 1 and are grouped together. What does forming that group of two achieve?

A. ☒ One input variable is eliminated from the term

B. ☐ The output of those cells becomes 0

C. ☐ Two input variables are eliminated

D. ☐ An extra term is added to the expression

Adjacent cells differ in exactly one variable, and that variable cancels when they are grouped. A group of two removes one variable; a group of four removes two.

7

Which of the following is not a valid Karnaugh map group?

A. ☐ A group of 1

B. ☐ A group of 2

C. ☒ A group of 3

D. ☐ A group of 4

Group sizes must be powers of two: 1, 2, 4 or 8. A group of three is never allowed.

8

A three-variable Karnaugh map has its columns labelled 00, 01, 11, 10. Which two columns, although drawn at opposite ends of the map, are treated as adjacent so that their cells can be grouped together?

A. ☐ 00 and 01

B. ☐ 01 and 11

C. ☒ 00 and 10

D. ☐ 11 and 10

In Gray code the end labels 00 and 10 differ in a single bit, so the map wraps and the outer columns are adjacent. Options The others are columns already drawn side by side, so they need no wrap-around.

9

Two students build different-looking circuits for the same task. What is the most reliable way to confirm the circuits are equivalent?

- A. ☐ Count the gates in each
- B. ☐ Compare the shapes of their Karnaugh map groups
- C. ☒ Check that their truth tables are identical
- D. ☐ Confirm they use the same operators

Two circuits implement the same function exactly when their truth tables match for every input combination. Gate count and operators can differ while the function stays the same.

10

Which expression is equivalent to NOT (A·B)?

- A. ☐ $\bar{A} \cdot \bar{B}$
- B. ☒ $\bar{A} + \bar{B}$
- C. ☐ $A + B$
- D. ☐ $\bar{A} \cdot B$

De Morgan's law turns NOT (A·B) into $\bar{A} + \bar{B}$: the operator changes from AND to OR and each variable is complemented. Option A is the common error of changing the operator but keeping AND, which actually equals NOT (A + B).

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**